

HY-QSN: HYbrid Quantum Safe Networks

Sayan Das

sayan005@e.ntu.edu.sg

Nanyang Technological University
Singapore

Prasanna Ravi

prasanna.ravi@ntu.edu.sg

Nanyang Technological University
Singapore

Aarav Varshney

aarav.varshney@ntu.edu.sg

Nanyang Technological University
Singapore

Anupam Chattopadhyay

anupam@ntu.edu.sg

Nanyang Technological University
Singapore

Abstract

A Quantum-Safe Network (QSN) enables secure quantum key exchange between consumers through interconnected Quantum Key Distribution (QKD) devices, overcoming the physical limitations of individual QKD systems. Critical sectors including telecommunications, defense, and finance access quantum-distributed keys via Quantum Key-as-a-Service (QKaaS) models. Current QKaaS implementations require both consumers to deploy QKD devices on their premises before establishing keys, creating cost and feasibility barriers that may limit adoption. While ETSI's GS QKD 014 standard defines APIs for QKD key retrieval, it does not address this deployment constraint. We identify this gap and propose a gateway-based solution that secures QKaaS models using Post-Quantum Cryptography (PQC). Our gateway transparently applies PQC-based signatures and key encapsulation mechanisms to ETSI-compliant QKD APIs without requiring infrastructure modifications. The solution supports cryptographic agility, enabling runtime switching between multiple PQC schemes. We validate our approach using both simulated QKD environments and production-grade hardware, demonstrating minimal overhead of 1.3x-1.5x compared to baseline performance. The solution scales effectively to production frameworks, successfully managing concurrent connections across multiple consumer sites.

Keywords

Quantum Safe Networks, Quantum Key Distribution, Post-Quantum Cryptography, Quantum-Safe Gateways, Cryptographic Agility

1 Introduction

Shor's algorithm [33], which prime-factorizes large integers in polynomial time, is becoming practical with large-scale, fault-tolerant quantum computers [34]. Ongoing research has led to projects like IBM Quantum Starling [20]—a large-scale, fault-tolerant quantum computer capable of executing 100 million quantum gates on 200 logical qubits by 2029. Given the rapid advancement of quantum computing capabilities, researchers are preparing for Q-Day [37], when public-key algorithms like RSA [32] and ECC (ECDHE and ECDSA) [1] used for key exchange and signatures will succumb to Cryptographically Relevant Quantum Computers (CRQCs) [38].

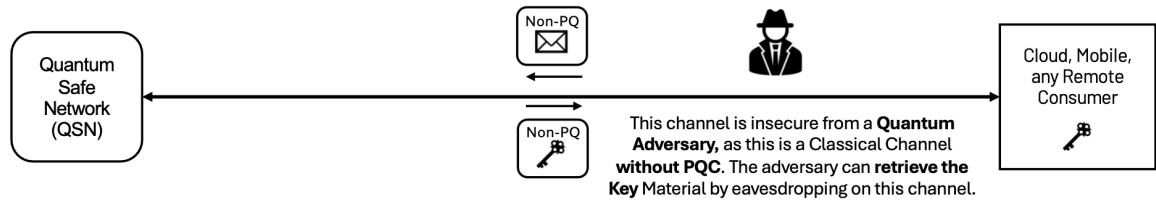
The cryptographic community has addressed the quantum threat by developing quantum-safe alternatives: Post-Quantum Cryptography (PQC) [2] and Quantum Key Distribution (QKD) [6]. PQC involves new cryptographic algorithms designed to resist quantum

attacks, while QKD leverages quantum properties of light for secure key exchange.

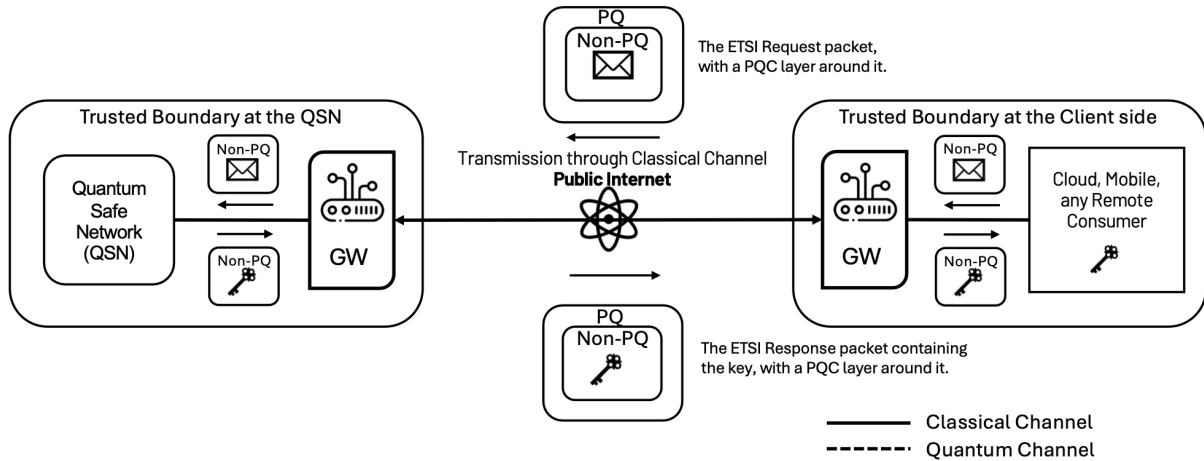
QKD enables two parties to establish a shared secret key by exchanging photons through optical fiber channels. The sender encodes random bits onto photons using quantum states (such as polarization), while the receiver measures these photons using randomly chosen measurement bases. Through subsequent classical communication, both parties compare a subset of their measurement bases to detect any eavesdropping—since quantum mechanics dictates that any interception attempt will disturb the photon states and introduce detectable errors [24]. The undisturbed photons yield a shared secret key with information-theoretic security. However, QKD faces significant physical constraints: photon loss increases exponentially with fiber distance, typically limiting secure key distribution to around 100-200 kilometers for fiber-based systems [16]. Additionally, QKD requires dedicated point-to-point optical infrastructure between each communicating pair.

To overcome these distance and scalability limitations, Quantum-Safe Networks (QSNs) [36, 39] interconnect multiple QKD devices through quantum channels, creating a distributed network for key distribution. QSN nodes act as trusted relays, performing quantum key exchange with their neighbors and then relaying keys across multiple hops to reach distant destinations. This network architecture extends the effective range of QKD beyond the physical limits of individual QKD links and enables many-to-many key sharing without requiring direct quantum connections between every communicating pair. Consumers access QKD keys from the network through locally deployed QKD devices connected to the QSN via quantum channels, with consumer-to-device communication occurring over private classical channels.

Although PQC dominates global standardization efforts, QKD is gaining traction through real-world deployments [21, 28, 41]. Government agencies and Banking, Financial Services and Insurance (BFSI) sectors are actively piloting QKD-based QSNs as core components of emerging quantum-safe infrastructure. However, current QKaaS models require each consumer to install and maintain a QKD device on their premises before participating in quantum key exchange, creating substantial entry barriers for organizations seeking quantum-safe solutions. This requirement limits QKD adoption to large enterprises and government entities, excluding smaller organizations, mobile users, cloud-based applications, and remote systems that could benefit from quantum-safe key distribution but cannot justify the infrastructure investment.



(a) Representation of a key-consumer retrieving QKD-keys via non-PQ algorithms over an insecure channel, where a quantum adversary can compromise the transferred key material.



(b) High-level overview of our approach using quantum-safe Gateways to wrap non-PQ packets with a PQ layer, enabling secure key transfer from the QSN to the consumer.

Figure 1: An overview of the problem statement identified (1a) and our proposed solution for the identified security gap (1b).

To address this limitation, consumers without local QKD devices could potentially access quantum keys through direct API interfaces with QSN nodes. The European Telecommunications Standards Institute (ETSI) [19] has developed the ETSI GS QKD 014 standard [18], defining standardized APIs for key-consuming applications, or Secure Application Entities (SAEs) in ETSI terms, to interface directly with QKD devices, or Key-Management Entities (KMEs) in ETSI terms. However, this standard uses classical cryptography and assumes that such interactions occur entirely within trusted environments, where consumer-to-QKD communication happens through channels with inherent security guarantees.

This assumption breaks down in practical QKaaS deployments where such trusted environments are not possible. In such scenarios, keys are delivered over public classical networks to remote consumers, as illustrated in Fig. 1a. When QKD keys traverse these classical channels using traditional TLS with non-post-quantum key exchange and authentication mechanisms, quantum adversaries can potentially intercept and decrypt the transmitted keys, negating the quantum-safe properties of the original QKD generation process.

This creates a critical security gap: while QKD provides information theoretic security for key generation and initial distribution within the QSN, the final delivery to consumers without local QKD devices relies on classical cryptography, vulnerable to quantum attacks. Furthermore, if post-quantum cryptographic protection is required to relay QKD-generated keys securely, consumers must also support PQC mechanisms. However, PQC adoption remains in early stages, with most organizations still evaluating these new standards. This leads to a fundamental question: who bears responsibility for ensuring PQC capabilities on the consumer side of QKaaS deployments?

In this paper, we propose a gateway-based solution, illustrated in Fig. 1b, that bridges this gap by inserting PQC-enabled gateways between QSNs and consumers using standard classical infrastructure. Our approach secures ETSI-compliant QKD interfaces using post-quantum cryptographic primitives without requiring changes to existing devices or software, thereby extending quantum-safe key delivery to consumers who cannot deploy local QKD infrastructure. Thus, we modify the current QSN model to a PQC-QKD hybrid QSN

model. We make our implementation and benchmarking scripts publicly available¹.

Our Contributions

The key contributions of our work are as follows:

- (1) **Standards Gap Identification:** We identify a critical mismatch between the ETSI GS QKD 014 [18] standard's assumption of trusted environments and the realities of QKaaS deployments, where key consumers often operate in untrusted or remote environments. This gap, addressed in detail in Section 4.1, has not been formally discussed in prior literature to our knowledge.
- (2) **Novel PQC-based Quantum-Safe Gateway Architecture:** We propose a gateway-based architecture that interposes a post-quantum cryptography layer between the QSN and classical key consumers, introducing the concept of hybrid QSNs. While the ETSI GS QKD 014 [18] protocol defines request and response packet formats, it does not mandate quantum-safe encryption, leaving transmissions exposed to quantum threats. Our design encapsulates both request and response packets with an additional PQC-secured layer using quantum-safe TLS between gateways. The gateway transparently secures QKD key delivery using PQC key encapsulation and digital signatures, ensuring quantum-safe transmission with minimal integration overhead.
- (3) **Cryptographic Agility and Runtime Flexibility:** Our gateway framework supports multiple PQC schemes and enables dynamic switching between them at runtime. This cryptographic agility [5] allows administrators to adapt to evolving security requirements and emerging post-quantum standards, including ML-KEM [26], ML-DSA [25], FrodoKEM [3], and BIKE [4], without modifying underlying infrastructure.
- (4) **Performance Evaluation at Scale:** We conduct extensive performance evaluations to quantify the latency overhead introduced by our gateway approach:
 - (a) *Simulated QKD Environments:* Using cloud-based QKD simulators, we assess the impact of geographical separation and various PQC schemes on key retrieval times, demonstrating system efficiency under practical conditions with overhead ranging from 1.3x to 1.5x.
 - (b) *Production QKD Hardware:* We validate the functional correctness and ease of integration of our gateway-based solution on a production-grade QKD hardware appliance.

We hope that our findings inform future revisions of the ETSI GS QKD 014 standard. In particular, we advocate for including guidance that explicitly addresses the need to protect interfaces between QSNs and key consumers using post-quantum cryptographic mechanisms. Such updates would acknowledge the practical realities of QKaaS deployments and strengthen end-to-end quantum-safe guarantees. We emphasize that our solution trades information-theoretic security for practical accessibility—while QKD provides unconditional security, our approach reduces security to computational assumptions inherent in post-quantum cryptography. This trade-off enables quantum-safe key distribution to consumers who

cannot deploy local QKD infrastructure, bridging the gap between theoretical quantum security and practical deployment constraints.

2 Literature Review

Recent advancements in quantum computing and QKD have catalyzed the development of various quantum-safe protocols and solutions, especially focused on securely distributing QKD-generated keys across adversarial networks. We provide a comprehensive review and comparative analysis of recent literature addressing quantum-safe solutions in cryptographic infrastructure.

Sergejs et al. [23] provide a similar framework for QKD-as-a-Service models, aiming to extend the availability of QKD-keys to remote consumers. They define their own set of protocols to achieve a quantum-safe key transport. However, the authors do not address the concept of QSNs and the ETSI standard in-place for QKD-key delivery from APIs. Our solution though following a similar approach, does not define new protocols and relies on the well-tested security properties of Transport Layer Security (TLS) v1.3 [15]. Moreover, the need for having systems with cryptographic agility has not been addressed in this research.

The QKDLite protocol [30] bridges the gap between the transient, point-to-point nature of QKD and the persistent key management needs of modern applications by integrating QKD infrastructures with Hardware Security Modules (HSMs). It supports a range of use cases—including store-and-forward protocols (e.g., secure email), permanent storage for file transfer, and periodic key usage scenarios (e.g., payment master keys)—by caching QKD- or QRNG-derived keys in HSMs [40] for subsequent retrieval and replication. Leveraging pQCee's pqTLS library, QKDLite enables quantum-safe communications that remain backward-compatible with classical TLS, allowing seamless deployment alongside existing IT systems. Each QKDLite node comprises modular components—such as the Transport Key Utility, QKDLite Utility, and QKDLite Scripts—and interfaces with HSMs, QKD Entities, and QRNG sources to perform key creation, export/import, and inter-node replication. Its flexible architecture supports diverse deployment configurations: from direct integration with quantum-safe TLS-capable appliances, to co-location with pqTLS for classical infrastructures, through digital QKD over VPN gateways, up to high-availability clusters for resilient multi-site replication. This versatility makes QKDLite well-suited for enterprise, IoT, and cloud-native environments requiring robust, scalable, and quantum-safe key distribution and lifecycle management.

Another solution QuProtect [12], developed by QuSecure, offers practical implementation of quantum-safe cryptographic protocols suitable for general-purpose network environments. The proposed architecture effectively ensures secure key exchanges in both trusted and partially trusted settings, but detailed gateway-based solutions with cryptographic agility are minimally addressed.

Bernstein et al. [10] propose PQConnect: Automated Post Quantum End-to-End Tunnels, a transparent tunneling protocol akin to VPNs but requiring no per-server configuration. PQConnect automatically discovers servers and establishes post-quantum IP tunnels using a conservative PQ-KEM (Classic McEliece [8]) for the server's long-term public key, supplemented with a secondary PQ-KEM (NTRUPrime [9]) for forward secrecy and classical elliptic-curve

¹<https://anonymous.4open.science/r/hybrid-qsn-asiaccs-E0C5>

fallback. The handshake security has been symbolically verified in Tamarin. Unlike our gateway-based architecture, PQConnect requires modification of the client and the server for direct end-to-end communication between the nodes, with limited focus on cryptographic agility.

NodeQ's PQTunnel [27] provides an automated post-quantum IP-tunneling framework that integrates seamlessly with existing network stacks. It employs a hybrid PQ-KEM design, combining lattice-based and code-based key encapsulation mechanisms, to achieve forward secrecy and minimal performance overhead, even on resource-constrained devices. PQTunnel dynamically discovers endpoints and establishes secure IP tunnels without manual per-host configuration, supporting live key rotation and end-point failover. While it excels at transparent end-to-end tunnel establishment in both enterprise and edge scenarios, it does not natively multi-entity key retrieval workflows as in our gateway-based solution.

We provide a comparative analysis of our solution with the existing solutions discussed in the literature in Table 1.

3 Preliminaries

This section provides the background necessary to understand the key concepts and challenges addressed in this paper. We cover Quantum Key Distribution (QKD), the ETSI GS QKD 014 Protocol [18], and real-world QKD deployments that form the basis of our proposed approach.

3.1 Quantum Key Distribution (QKD)

Quantum Key Distribution (QKD), proposed by Bennett et. al. [7] in 1984 through the BB84 protocol, and verified by Shor et. al. [35] in 2000, uses the principles of quantum mechanics to transmit data in the form of photons. Unlike classical key exchange protocols that rely on computational hardness assumptions, QKD encodes key bits into quantum states, through photon polarization, and transmits them over fiber optic channels. Due to fundamental quantum properties like the no-cloning theorem [44] and Heisenberg's uncertainty principle [11], any eavesdropping attempt introduces detectable disturbances, alerting the communicating parties to potential security breaches. QKD thus offers security resistant to both classical and quantum adversaries, providing unconditionally secure keys for subsequent encryption tasks.

While QKD successfully addresses secure key generation and exchange, integrating these keys into traditional security infrastructures presents significant challenges [16]. QKD-keys must be consumed by existing encryption devices and applications—such as IPsec VPN gateways, TLS libraries, or MACsec appliances—which were not originally designed to interface with quantum sources. This disconnect has created a need for standardized methods of QKD-key retrieval and usage. International bodies such as the European Telecommunications Standards Institute (ETSI) have developed protocols like ETSI GS QKD 014 [18] to define uniform interfaces for accessing QKD-generated keys, bridging the gap between quantum-secure key generation and real-world secure communication systems.

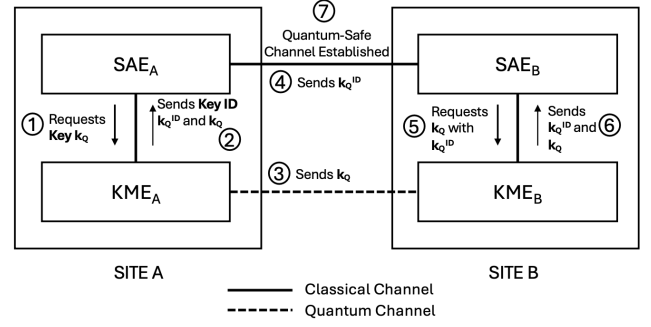


Figure 2: Trusted-boundary setup under ETSI GS QKD 014 where Steps 1 to 7 describe how SAE_A initiates communication with SAE_B through KME_A and KME_B respectively, establishing a QKD-key k_Q for secure site-to-site communication.

3.2 The ETSI GS QKD 014 Protocol

The ETSI GS QKD 014 standard [18] defines specifications for a secure key delivery API used by QKD devices. This protocol, described in Fig. 2, is part of ETSI's broader effort to standardize QKD integration into conventional cryptographic infrastructure. The protocol outlines how a Secure Application Entity (SAE), formally defined as an entity that can request and fetch QKD-generated keys from a Key Management Entity (KME), which is formally defined as an entity that manages the keys in a QSN in cooperation with one or more KMEs, using a RESTful API.

The protocol defines three main operations: *Key Request*, where the SAE requests a quantum-generated key from the KME by specifying parameters such as key length, peer identities, and key identifiers; *Key Retrieval*, where the KME securely delivers the requested key or returns an error if unavailable; and *Metadata Management*, which allows querying key metadata, checking key generation status, and managing key lifetimes or destruction. The cryptographic assumptions are that the SAE and KME operate within the same trusted environment over a private channel inaccessible to adversaries, that their communication is secured using TLS 1.2 or higher, and that the KME only releases keys upon validating the SAE's identity via TLS certificates.

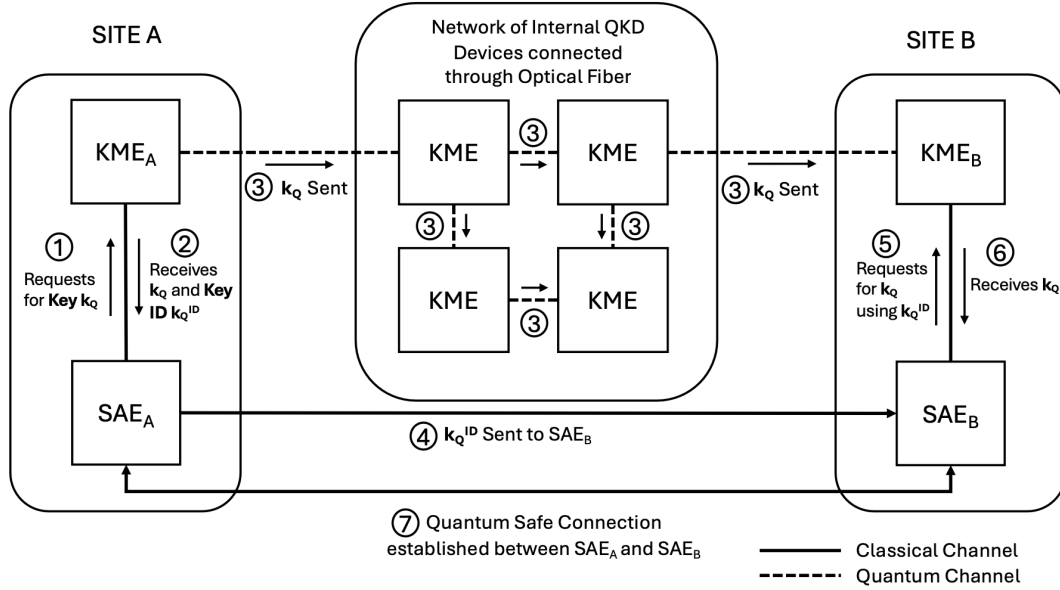
Examples of SAEs include devices that utilize QKD-derived keys for subsequent encryption tasks, such as network encryptors and other cryptographic appliances responsible for securing data in transit.

3.3 Quantum Safe Network (QSN) Using QKD Devices

A Quantum Safe Network (QSN) is defined as a network of interconnected KMEs, that generate and transport QKD-keys from one site to another through the quantum channel. Fig. 3 outlines the high-level architecture for a quantum-safe key distribution network that combines dedicated fiber-based QKD links (quantum channels). The setup includes a SAE-KME pair for each site. As shown in Fig. 3, it is to be noted that SAE_A operates within a trusted enclave alongside Site A's KME_A; SAE_B is similarly co-located with the KME_B at Site B.

Table 1: Comparison of Our Solution with Existing Quantum-Safe Solutions

Feature / Solution	QaaS [23]	QKDLite [30]	QuProtect [12]	PQConnect [10]	PQTunnel [27]	Our Solution
Deployment Model	QKD-as-a-Service; custom protocols for remote QKD key delivery	Modular architecture with HSM integration; flexible topologies (direct, co-located, VPN, clustered)	General-purpose network security platform, limited details on gateways	VPN-like PQ tunnels; requires client/server modification	Automated PQ IP tunnels; transparent integration with stacks	Dual quantum-safe gateways forming overlay tunnel between QSN and consumer
Cryptographic Mechanism	Custom PQ protocols (not standardized)	pqTLS with cached QKD-QRNG-derived keys via HSM	Proprietary PQ cryptographic suite (details abstracted)	Hybrid KEM: Classic McEliece + NTRUPrime + ECC fallback	Hybrid lattice- and code-based KEM design	Configurable PQC primitives: ML-KEM, HQC, BIKE, ML-DSA, SLH-DSA
Cryptographic Agility	Not addressed	Limited to pqTLS library capabilities	Minimal; not the main focus	Conservative choice with fallback; limited agility	Fixed hybrid design; no runtime selection	Full agility through configuration controller supporting multiple KEMs/DSAs
End-to-End Quantum Safety	Yes (custom PQ-based protocols)	Yes (via TLS-based protection)	Yes (general PQ-protected exchanges)	Yes (post-quantum tunnels with verified handshake)	Yes (tunnel-based PQ protection)	Yes, even when consumer endpoints are legacy/non-PQ
Supports Existing Infrastructure	No; requires adherence to new protocol definitions	Works with pqTLS-enabled appliances and HSMs; integration needed	Designed for enterprise networks, limited documentation on legacy support	Requires endpoint modification, not transparent	Seamless drop-in at network stack level	Fully interoperable; no endpoint changes required
Cryptographic Load at Endpoint	Moderate to high (depends on their custom protocols)	Moderate (TLS cryptographic cost)	Potentially high (endpoints handle PQ)	High (PQ handled directly by endpoints)	High (tunnel endpoints perform PQ operations)	Low; cryptographic operations offloaded to gateways
Standards Compliance	Not aligned with ETSI GS QKD 014; custom design	ETSI-compliant via pqTLS + HSM	Not explicitly mentioned	Custom PQ tunneling protocol	Not emphasized	Wraps PQ layer over TLS 1.3 and ETSI GS QKD 014 workflows
Unique Contributions	Defines new framework for QKD-as-a-Service; extends key delivery to remote consumers	Persistent QKD-QRNG key lifecycle management via HSM caching	Practical enterprise PQ security platform; orchestration focus	Automated PQ tunnel setup with Tamarin-verified handshake	Lightweight, dynamic PQ tunneling with live key rotation and failover	Cryptographic-agile dual-gateway overlay enabling transparent QSN-to-consumer key delivery

**Figure 3: The QSN architecture demonstrating the use of QKD to establish a secure connection between two SAEs at different locations. Steps 1 to 7 indicate the information flow through the setup.**

- **Key Generation at Site A:** The Key Management Entity (KME_A) at Site A generates quantum key k_Q and its API returns the pair (k_Q, k_Q^{ID}) to the Secure Application Entity (SAE_A) at Site A.
- **Transport of QKD-Key (k_Q) through Internal KMEs:** The chain of fiber-linked QKD devices (the internal QKD

framework) securely relays k_Q from Site A to Site B through the quantum channel.

- **Key ID (k_Q^{ID}) Notification and Transfer:** Upon retrieving (k_Q, k_Q^{ID}) , SAE_A transmits the identifier k_Q^{ID} to SAE_B over an authenticated classical channel.

- **Key Retrieval at Site B:** SAE_B uses the received k_Q^{ID} to query its local KME_B and obtains the identical key k_Q .
- **Mutual Authentication:** Every SAE - KME interaction employs the mutual-authentication handshake defined in ETSI GS QKD 014; any authentication failure aborts the protocol.

3.4 Real World Deployments of QKD

The growing threat of quantum computing to classical cryptographic systems has led to a global surge in the research, development, and deployment of Quantum-Safe Networks (QSNs). Governments, research institutions, and industry leaders are increasingly investing in QKD technologies and quantum-safe encryption protocols to protect critical infrastructure. Several real-world deployments highlight the progress made in this domain:

- **South Korea:** Nokia and SK Broadband have jointly deployed a quantum-safe network to secure Korea Hydro and Nuclear Power's (KHNP) IT infrastructure [28]. This deployment utilizes Nokia's Quantum-Safe MACsec encryption technology over IP/MPLS networks to defend against quantum-enabled cyber threats, forming a secure leased line based on standards-compliant quantum-safe cryptography.
- **Singapore:** The Infocomm Media Development Authority (IMDA) has launched Southeast Asia's first National Quantum-Safe Network Plus (NQS+ [21]. This initiative enables quantum-safe communications across the nation's digital infrastructure, supporting both enterprise adoption and research into scalable quantum-safe key exchange mechanisms.
- **Industry Collaboration:** Juniper Networks, in collaboration with ID Quantique and Deutsche Telekom, validated the integration of Quantum-Safe MACsec using QKD-based keys in a multivendor environment [22]. This proof-of-concept tested interoperability based on the ETSI QKD REST API specification [18], focusing on securing MACsec-enabled links while addressing practical deployment considerations.
- **European Initiatives:** Quantum Delta NL's Quantum Network (QN) [31] offers the Netherlands' first national physical network connecting multiple quantum processors via both fiber and satellite links. The UK National Quantum Technologies Programme (NQTP) [42] has committed £1 billion to foster a quantum-enabled economy, with its Quantum Communications Hub demonstrating live QKD links at the 2024 Farnborough International Air Show.

These deployments and research initiatives show the increasing maturity of QKD technology and its integration into commercial applications. VentureRadar catalogs dozens of QKD-focused companies [43], providing innovation scores and funding insights that guide procurement and R&D in quantum communications.

4 Motivation

Organizations are increasingly integrating QKD-keys into their cryptographic infrastructures to achieve information-theoretic security. The ETSI GS QKD 014 protocol [18] assumes that the SAE and KME exist within the same trusted boundary, ensuring protected key exchanges. However, real-world deployments are rapidly evolving beyond this model.

4.1 Expanding the Trusted Boundary

The identified gap between the QKaaS model and ETSI GS QKD 014 standard is explained by referencing the QSN architecture (Fig. 3) and considering a scenario, stated in Fig. 4, where the SAE_B does not have a KME within its trusted boundary. This is a valid problem as deploying QKD devices (KME s in this context) at each consumer's site incurs a very high cost and is not feasible due to the constraint of establishing a quantum channel between the QSN and the consumer's site. There might be a scenario when the KME or the quantum channel breaks down in the consumer's site, resulting in stoppage of services at the consumer's end. Also, the consumers of QKD-keys might be diverse. Let's consider a scenario where a financial institution wants to use QKD-keys to encrypt the communication between its central server and its numerous ATMs. The ATM wants to establish a QKD-key with its banking server located somewhere in a secure site for communication. It's practically impossible for each ATM to have a KME of its own, or for the bank to lay fiber-links for QKD to each ATM. These considerations pose a critical question:

How can QKD-keys be securely delivered from the QSN to the SAE when the receiving site lacks a KME within its trusted boundary?

We address this challenge with a gateway-based solution that ensures the use of post-quantum cryptographic (PQC) primitives for secure key material transport, as detailed in Section 5.2.

4.2 Cryptographic Agility and Long-term Security

With evolving quantum threats, a critical concern remains:

How can we ensure the long-term security and adaptability of our solution if today's post-quantum cryptographic standards are compromised?

This concern becomes particularly acute when considering scenarios where quantum algorithms might break currently deployed lattice-based cryptographic schemes, as evident from the progress on recent attacks on lattice-problems [13]. Organizations would need to rapidly transition to alternative cryptographic primitives, potentially causing significant operational disruption. To address this challenge, our solution incorporates robust cryptographic agility, allowing seamless algorithm substitution without architectural changes. We demonstrate how our gateway-based design facilitates rapid adaptation to emerging cryptographic standards while maintaining continuous protection of key transport mechanisms in Section 6.

5 PQC-enabled Gateways for Secure Key Transport

We propose a gateway-based solution that secures communication between KME s and SAE s by adding a post-quantum cryptographic layer. Our approach uses transport-layer gateways that intercept and relay traffic while applying cryptographic transformations, operating transparently without requiring changes to existing SAE or KME configurations.

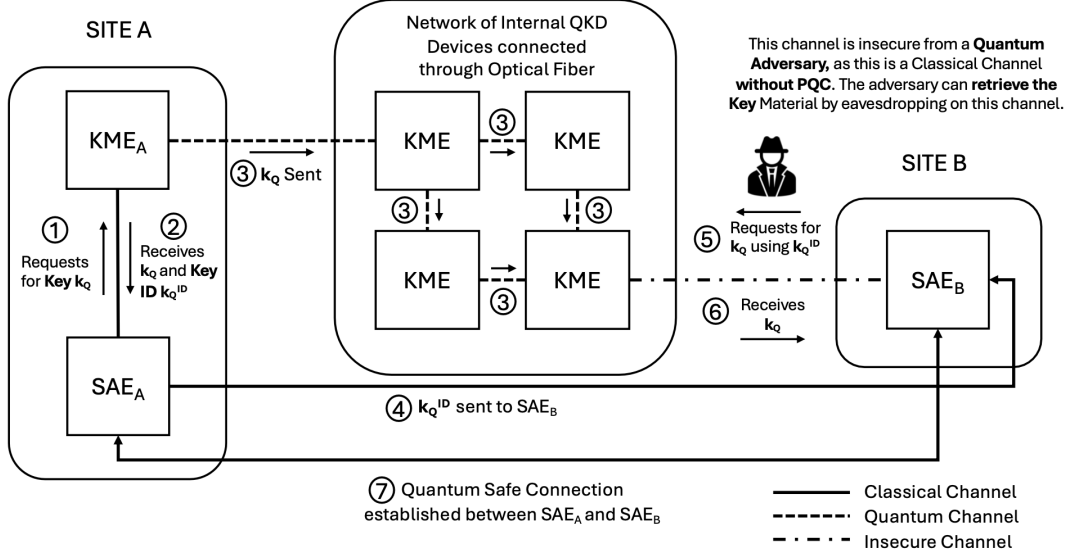


Figure 4: This figure points out the problem if a local Key Management Entity (KME) is not available at Site B, but SAE_B wants to establish a connection and share data with SAE_A from Site A. SAE_B will need to fetch the key from a KME in the QSN, where the trusted boundary assumption breaks, and the channel between the SAE_B and the KME it is fetching the key from is not private and is insecure against a quantum adversary.

5.1 Gateway Architecture and Operation

Our solution employs a dual-gateway architecture where a server-side gateway and a client-side gateway are deployed at the SAE and the KME side, respectively (see Fig 5). The key operational principle is straightforward: maintain full ETSI GS QKD 014 API compatibility while upgrading the underlying transport security from classical TLS to PQC-enabled TLS. We briefly outline the packet flow process as follows:

- (1) **Client-side Processing:** When the SAE initiates a key request using the standard ETSI GS QKD 014 API format, the client-side gateway intercepts the outgoing packet (originally destined for classical TLS transmission) and establishes a quantum-safe TLS 1.3 connection with the server-side gateway using post-quantum key exchange algorithms.
- (2) **Secure Tunnel Establishment:** The gateways perform mutual authentication using quantum-safe digital signatures, creating an authenticated and encrypted tunnel that ensures both integrity and confidentiality of the communication channel.
- (3) **Packet Encapsulation:** The original ETSI GS QKD 014 API request—maintaining its standard format for compatibility—is encapsulated within the PQC-secured TLS layer. Since the ETSI standard was not designed to use PQC and KMEs may not support ETSI requests over PQC directly, our gateways simply wrap the classical TLS-formatted requests in an additional PQC TLS layer.
- (4) **Server-side Processing:** Upon reaching the server-side gateway, the PQC encryption layer is removed and the original ETSI-compliant API request is forwarded to the KME over classical TLS as expected by the standard.

5.2 Protocol Description

Notations. We define the notations for our protocol stated as follows:

- π_{NPQ} : Represents a classical TLS handshake between SAE_B and the KME using classical key exchange and authentication schemes to establish symmetric session key k_S .
- π_{PQ} : Represents a post-quantum TLS handshake between the gateways (GW) using post-quantum key exchange and authentication schemes to establish symmetric session key k_{PQ} .
- $Enc_k(pt)$: Represents the encryption operation using a symmetric cipher based on the chosen TLS ciphersuite on a plaintext pt using symmetric key k .
- $Dec_k(ct)$: Represents the decryption operation using a symmetric cipher based on the chosen TLS ciphersuite on a ciphertext ct using key k .
- $GetKey(k^{ID})$: Represents the ETSI compliant key request sent by the SAE_B to the KME using the key ID k^{ID} .

The sequence diagram provided in Fig. 6 describes our protocol, where gateways are used as middleware between the SAE_B and the KME within the QSN, to transport the key in a quantum-safe manner. Our proposed solution comes in during execution of Steps 5 and 6 from the diagram in Fig. 5.

Information Flow in the Protocol. Figure 6 illustrates the end-to-end message flow for fetching a QKD key (k_Q) at SAE_B when no local QKD endpoint is available. The entities involved are SAE_B , two quantum-safe gateways (GW), and KME. The protocol proceeds in two nested handshake phases followed by an encrypted application request and response:

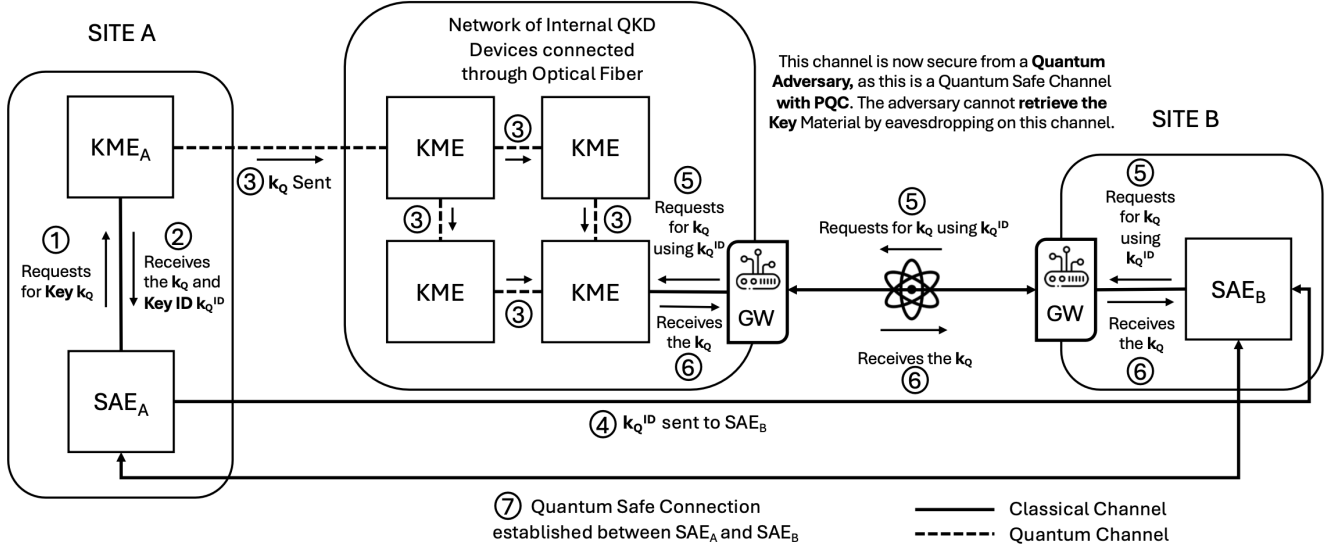


Figure 5: Extension of QKD-key reachability using our proposed quantum-safe gateways, enabling SAE_B at Site B to securely obtain keys from the QSN even without a local KME.

- (1) **Outer Classical TLS Handshake:** SAE_B initiates a classical TLS handshake (π_{NPQ}), through the gateways, with the KME using classical key exchange and certificates. This establishes the session key k_S between the SAE_B and the KME, and all encrypted messages traverse the quantum-safe tunnel managed by the gateways.
- (2) **Inner Post-Quantum TLS Handshake:** Upon receiving a key request from the SAE_B , the two gateways perform a post-quantum TLS 1.3 handshake (π_{PQ}) using post-quantum key exchanges and digital signatures for mutual authentication. This creates the symmetric quantum-safe key k_{PQ} , ensuring confidentiality and integrity of all tunneled messages.
- (3) **Tunneling of Handshake Messages:** Once k_{PQ} is established, the classical ClientHello/ServerHello messages of the outer classical TLS handshake are encrypted with k_{PQ} as $Enc_{k_{PQ}}(\pi_{NPQ})$ and forwarded through the gateways. At each hop, the gateways decrypt and re-encrypt only the messages under k_{PQ} before delivery, preserving the end-to-end integrity and confidentiality of π_{NPQ} .
- (4) **Application-Level Key Request:** After both handshakes complete, SAE_B issues an ETSI-compliant key request encrypted under k_S as $Enc(\text{GetKey}(k_Q^{ID}))$. The gateway at SAE_B end, encrypts the ETSI-compliant key request using k_{PQ} . Upon decrypting the outer layers, KME authenticates the request and sends the QKD-key to the SAE_B through the gateways.
- (5) **Application-Level Key Delivery:** The KME returns k_Q and its identifier k_Q^{ID} encrypted under k_S . The server-side gateway encrypts this response under k_{PQ} , forwarding it to the client-side gateway, which decrypts the PQ layer and forwards the response to the SAE_B , which decrypts the response using k_S to get the QKD-key k_Q .

- (6) **Authentication Guarantees:** All handshake messages in π_{NPQ} and π_{PQ} involve mutual authentication of endpoints (either via classical or post-quantum certificates), ensuring that each party is verified before key material is exchanged.

This dual-layer approach combines the proven interoperability of classical TLS with the quantum resistance of post-quantum TLS, enabling seamless integration into existing infrastructures while safeguarding against both classical and quantum adversaries.

5.3 Security

Our security analysis relies on the established security properties of TLS 1.3 [15] with post-quantum cryptography to protect the communication channel between gateways.

Security Properties. We define three core security properties for our protocol:

- **Correctness:** A QKD key generated at a KME is successfully delivered to the requesting SAE.
- **Confidentiality:** Adversaries cannot learn the transmitted QKD key from network traffic.
- **Authentication:** Only authenticated SAEs can retrieve QKD keys from the KME.

Assumptions. We assume the following:

- (1) **Trusted Internal Channels:** Channels between $SAE_B \leftrightarrow GW$ and $GW \leftrightarrow KME$ occurs over trusted internal networks.
- (2) **Adversarial Public Channel:** The inter-gateway communication traverses untrusted networks where adversaries can intercept, modify, or inject traffic.
- (3) **TLS Security:** Both classical TLS 1.2 and post-quantum TLS 1.3 provide their standard security guarantees (authentication, confidentiality, integrity).

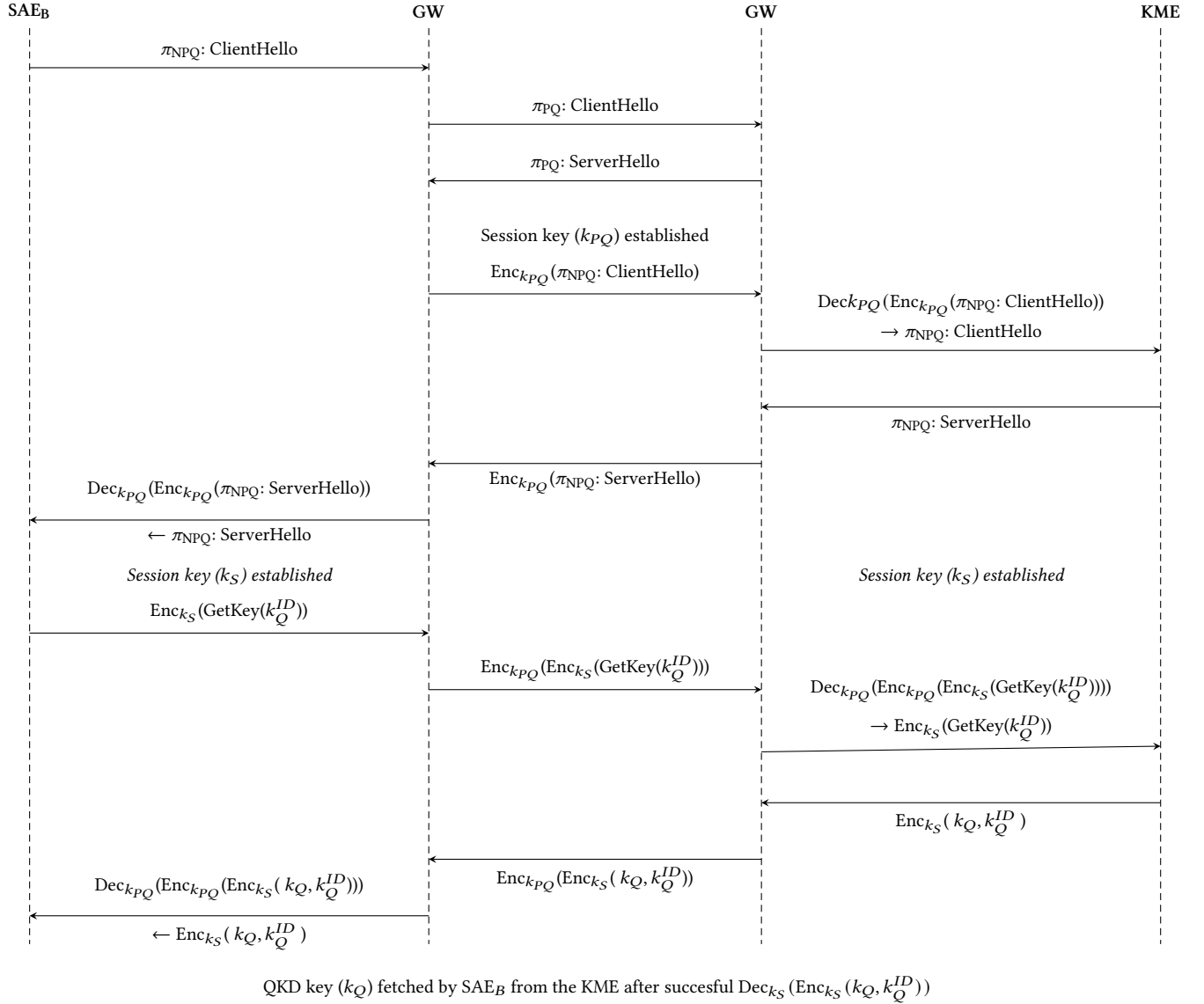


Figure 6: The Protocol Diagram stating the transfer of a key in the absence of a KME at SAE_B's end. The introduction of quantum-safe Gateways (GW), helps in fetching the QKD-key (k_{Q}) in a quantum-safe manner.

Security Arguments. Our protocol achieves security through a dual-layer approach:

- **Inner Layer (PQ-TLS 1.3):** The post-quantum TLS tunnel between gateways protects against quantum adversaries on the public channel. All traffic—including the outer TLS handshake and key delivery messages—is encrypted under the PQ tunnel key k_{PQ} . This ensures that even quantum-capable adversaries observing network traffic cannot decrypt the communication.
- **Outer Layer (Classical TLS 1.2):** The standard ETSI compliant TLS connection between SAE and KME provides end-to-end authentication and message integrity. The KME only

releases keys after authenticating (via TLS certificates) the requesting SAE through this channel.

- **Combined Security:** An adversary seeking to compromise the protocol would need to break both the post-quantum TLS tunnel (to access encrypted traffic) and the classical TLS authentication (to impersonate legitimate SAEs). Since these layers use different cryptographic primitives, the protocol maintains security even if one layer is compromised.

The gateway approach preserves all existing ETSI security mechanisms while adding quantum resistance to the transport layer, ensuring that QKD keys remain secure during transmission over untrusted networks.

6 Implementation

To evaluate the performance impact of our PQC gateway solution, we use several mature open-source components:

- **Envoy Proxy:** We deploy quantum-safe gateways as transparent (i.e., without modifying the original TLS request) TLS interceptors using Envoy's high-performance L3/L4 proxy capabilities [17].
- **oqs-envoy:** Post-quantum cryptographic support in Envoy was achieved through a custom Envoy build integrated with liboqs [29], the OQS cryptographic library.
- **Kubernetes:** Kubernetes [14] handles the gateway orchestration and lifecycle management. It helps in automating deployment on both the SAE and the KME sites as well as scaling, and configuration management. The declarative API enables dynamic cryptographic agility through rolling updates of gateway instances without service interruption. In our proof-of-concept, we update the configuration file to limit the key exchange curves to a specified PQC KEM.

Our experimental setup is shown in Fig 7. The setup consists of two Amazon EC2 instances representing distributed SAE and KME environments. On the SAE side, a client gateway co-located on the same EC2 instance intercepts outbound ETSI GS QKD 014 API calls and establishes PQC-protected connections to the server gateway. The server gateway, deployed as a Kubernetes service with two Envoy proxy pods, terminates the PQC connection and forwards requests to the KME simulation layer implemented using Nginx. While the Kubernetes controller manages KEM agility by consuming the configuration files, signature agility was provided using updated file mounts, as Kubernetes allowed direct import of PQC certificates.

6.1 Experimental Results

We conduct performance benchmarking to quantify the computational and latency overhead introduced by the PQC gateway layer. We used wrk HTTP benchmarking tool configured with 4 worker threads across concurrency levels ranging from 10 to 500 simultaneous connections, with each test sustained for 30 seconds to capture steady-state performance characteristics. Three post-quantum algorithm combinations were evaluated: ML-KEM-1024 + ML-DSA-87, BIKE-L5 + ML-DSA-87, and FrodoKEM-1344-SHAKE + ML-DSA-87, representing different cryptographic approaches. The baseline configuration utilized direct Nginx TLS 1.2 connections without the gateway layer to isolate the PQC-specific performance impact.

Experimental results demonstrate that our PQC gateway solution provides quantum-resistant protection while maintaining offering similar throughput performance compared to our baseline (Fig. 8). The ML-KEM-1024 + ML-DSA-87 combination achieved a better performance among PQC variants with a robust peak throughput of 5,738 requests per second, operating at 74% of baseline performance. BIKE-L5 + ML-DSA-87 and FrodoKEM-1344-SHAKE + ML-DSA-87 demonstrated strong performance characteristics, maintaining 68% and 67% of baseline throughput respectively, confirming the viability of diverse cryptographic approaches. Latency analysis reveals that PQC variants add only 0.38-0.60ms overhead (1.56-1.78ms versus 1.18ms baseline), demonstrating minimal impact on user experience. Under high-stress conditions, the gateway maintained

stable operation while the additional security processing introduced measured latency increases consistent with the cryptographic protection provided. We have provided detailed results using multiple algorithms for different concurrency levels in the Appendix (A).

6.2 Experimenting on QKD Hardware

We have validated our gateway-based solution end-to-end using a production-grade QKD hardware appliance. We confirmed seamless compatibility between our proposed gateways and the QKD devices, successfully retrieving QKD keys under real-world optical link conditions. Experiments are currently ongoing to measure performance metrics such as key throughput, latency, and reliability. As these benchmarking efforts are not yet complete, we defer detailed numerical results to a future version of this paper, wherein we will present a comprehensive evaluation of system performance under various network conditions.

6.3 Feasibility and Scalability

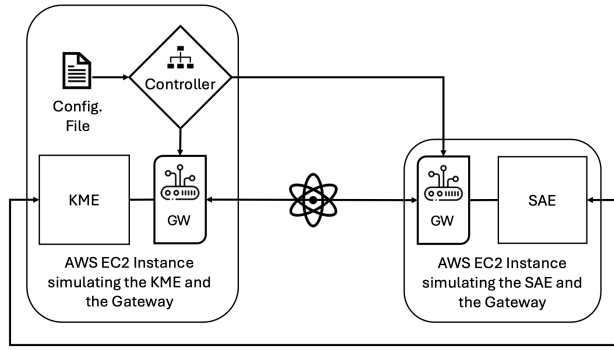
Our gateway-based design for a hybrid QSN was simulated (Fig. 7) using AWS infrastructure. Kubernetes enabled runtime cryptographic agility and zero-downtime rollouts. Benchmarks (Fig. 8) show that the PQC gateway layer sustains high throughput with minimal overhead. Multi-client experiments, with several client gateways concurrently connected to a single server gateway, preserved functional correctness and authentication while scaling cleanly under Kubernetes with state-of-the-art request management for stability. Finally, end-to-end tests against production-grade QKD hardware validated interoperability and key retrieval. Collectively, these results attest that our approach is feasible, operationally robust, and performance-viable for near-term deployment while offering cryptographic agility as PQC standards evolve. We make our experimental data and codebase for the gateways available on this link².

7 Conclusion and Future Work

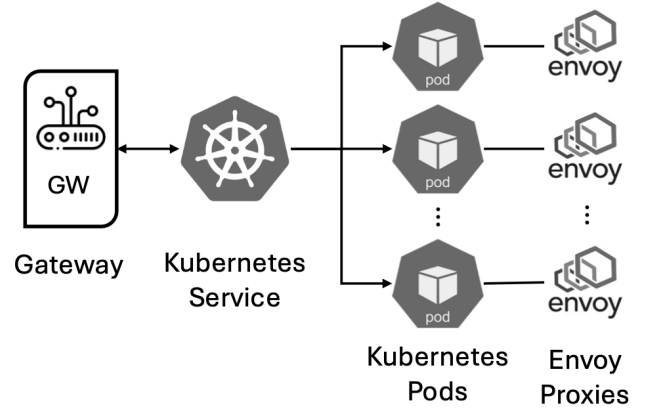
As organizations migrate toward quantum-safe architectures, Quantum Key-as-a-Service (QKaaS) exposes classical applications to QKD-derived keys via standardized interfaces (e.g., ETSI GS QKD 014). In heterogeneous, real-world networks, however, QKD endpoints are not always co-located with the consuming applications.

This paper tackles a practical gap in QKaaS deployments: when a local KME is unavailable to the consumer, the standard request and response flow cannot proceed, leaving applications without a path to obtain QKD-keys from QSNs. We introduced a gateway-based design that bridges this gap by fetching keys from a QSN over classical networks secured with post-quantum cryptography—an inner TLS 1.3 PQ tunnel carrying the unmodified outer TLS 1.2 application channel—so existing SAE/KME software remains unchanged. Our implementation on Envoy (with oqs-boringssl) and Kubernetes demonstrated runtime cryptographic agility, multi-tenant operation (many client gateways to one server gateway), and interoperability with production-grade QKD hardware. Benchmarks indicate that efficient PQC choices (e.g., ML-KEM with ML-DSA)

²<https://anonymous.4open.science/r/hybrid-qsn-asiaccs-E0C5>



(a) Our experimental setup on AWS consists of two separate instances each hosting a gateway. The gateways are managed by a Kubernetes controller and facilitate communication between the SAE and the KME.



(b) Internal structure of our gateways. Each gateway runs a Kubernetes service running multiple pods which use our post-quantum enabled Envoy proxies.

Figure 7: Implementation of our solution using AWS EC2 instances, running gateways using Kubernetes Service, multiple pods and envoy proxies underneath.

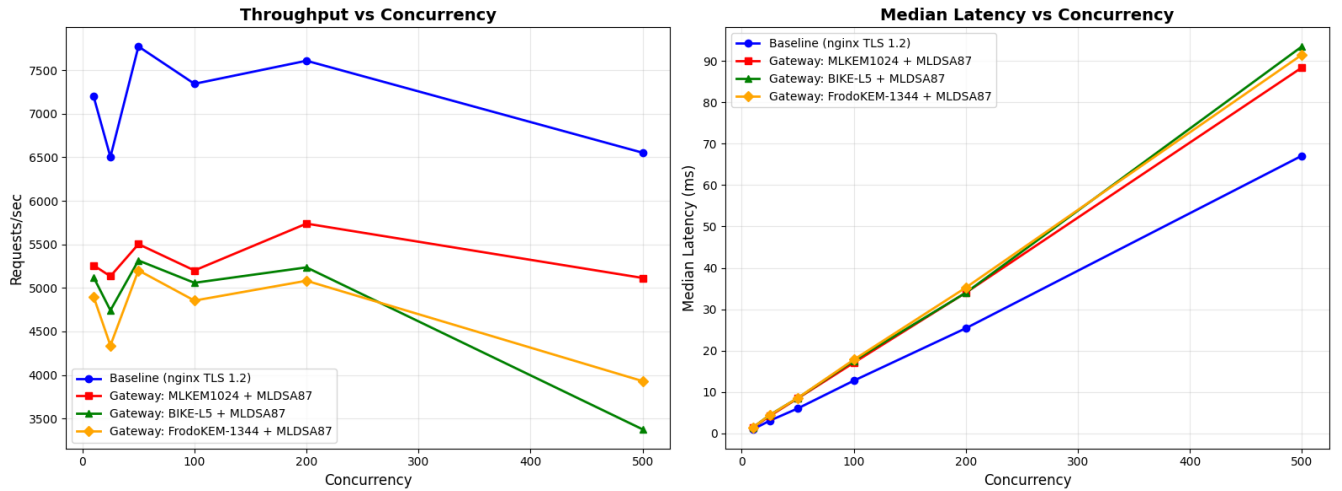


Figure 8: The graphs shown provide a comparison of the overhead incurred by our solution over a simple TLS 1.2 key transfer, using different quantum-safe key exchange algorithms, namely ML-KEM, BIKE, FrodoKEM, and ML-DSA as the signature algorithm. The first plot on the left shows a throughput based on the number of concurrent requests, while the plot on the right provides the median latency based on the number of concurrent connections.

deliver strong security with modest throughput and latency overheads, supporting the viability of the approach for QKaaS scenarios where the consumer lacks a co-located KME.

Future Work. Future work will broaden beyond implementation to a more general agenda: deploying our solution on a QSN and analyzing security at the system level with formal arguments for authenticated key release and end-to-end correctness; studying fairness and quality-of-service for many consumers sharing a QSN node to guide large-scale, real-world adoption.

References

- [1] 2004. *Elliptic Curve Arithmetic*. Springer New York, New York, NY, 75–152. doi:10.1007/0-387-21846-7_3
- [2] 2025. Post-Quantum Cryptography. <https://csrc.nist.gov/projects/post-quantum-cryptography> Accessed: 2025-04-30.
- [3] Erdem Alkim, Joppe W. Bos, Leo Ducas, Paulo Longa, Ilya Mironov, Michael Naehrig, Vitaly Nikolaenko, Chris Peikert, Arjun Raghunathan, and Douglas Stebila. 2017. FrodoKEM: Learning With Errors Key Encapsulation. <https://frodokem.org>. Accessed: 2025-08-07.
- [4] Nicolas Aragon, Paulo Barreto, Slim Bettaieb, Loic Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Santosh Ghosh, Shay Gueron, Tim Guney, et al. 2022. BIKE: bit flipping key encapsulation.
- [5] Elaine Barker, Lily Chen, David Cooper, Dustin Moody, Andrew Regenscheid, Murugiah Souppaya, William Newhouse, Russ Housley, and Sean Turner. 2025.

- Considerations for Achieving Cryptographic Agility: Strategies and Practices (Draft)*. Technical Report. US Department of Commerce.
- [6] Charles H. Bennett and Gilles Brassard. 2014. Quantum cryptography: Public key distribution and coin tossing. *Theoretical Computer Science* 560 (Dec. 2014), 7–11. doi:10.1016/j.tcs.2014.05.025
 - [7] Charles H Bennett and Gilles Brassard. 2014. Quantum cryptography: Public key distribution and coin tossing. *Theoretical computer science* 560 (2014), 7–11.
 - [8] Daniel J Bernstein, Tung Chou, Tanja Lange, Ingo von Maurich, Rafael Misoczki, Ruben Niederhagen, Edoardo Persichetti, Christiane Peters, Peter Schwabe, Nicolas Sendrier, et al. 2017. Classic McEliece: conservative code-based cryptography. *NIST submissions* 1, 1 (2017), 1–25.
 - [9] Daniel J Bernstein, Chitchanok Chuengsatiansup, Tanja Lange, and Christine van Vredendaal. 2019. NTRU Prime: round 2. *Submission to the NIST PQC standardization process*, URL: <https://ntruprime.cr.yp.to> (2019).
 - [10] Daniel J Bernstein, Tanja Lange, Jonathan Levin, and Bo-Yin Yang. 2024. PQConnect: Automated Post-Quantum End-to-End Tunnels. *Cryptology ePrint Archive* (2024).
 - [11] Paul Busch, Teiko Heinonen, and Pekka Lahti. 2007. Heisenberg's uncertainty principle. *Physics reports* 452, 6 (2007), 155–176.
 - [12] Chris Cap, Scott Kawaguchi, Joey Lupo, and Chris Trinidad. 2024. Secure Communications for Quantum and Beyond: Orchestrated Cryptography for Agility and Policy. In *Counterterrorism and Cybersecurity: Total Information Awareness*. Springer, 569–584.
 - [13] Yilei Chen, Qipeng Liu, and Mark Zhandry. 2021. Quantum Algorithms for Variants of Average-Case Lattice Problems via Filtering. arXiv:2108.11015 [quant-ph] <https://arxiv.org/abs/2108.11015>
 - [14] Cloud Native Computing Foundation. 2024. Kubernetes Documentation. <https://kubernetes.io/docs/home/>. Accessed: 2025-08-07.
 - [15] Cas Cremers, Marko Horvat, Jonathan Hoyland, Sam Scott, and Thyla van der Merwe. 2017. A Comprehensive Symbolic Analysis of TLS 1.3. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security* (Dallas, Texas, USA) (CCS '17). Association for Computing Machinery, New York, NY, USA, 1773–1788. doi:10.1145/3133956.3134063
 - [16] Eleni Diamanti, Hoi-Kwong Lo, Bing Qi, and Zhiliang Yuan. 2016. Practical challenges in quantum key distribution. *npj Quantum Information* 2, 1 (2016), 1–12.
 - [17] Envoy Project. 2024. Envoy Proxy Version 1.35.0 Documentation. <https://www.envoyproxy.io/docs/envoy/v1.35.0/>. Accessed: 2025-08-07.
 - [18] ETSI. 2019. Quantum Key Distribution (QKD); Protocol and data format of REST-based key delivery API. https://www.etsi.org/deliver/etsi_gs/QKD/001_099/014/01.01.01_60/gs_qkd014v010101p.pdf. Accessed: 2025-04-30.
 - [19] ETSI. 2025. ETSI. <https://www.etsi.org/>. Accessed: 2025-04-30.
 - [20] IBM Quantum. 2025. How IBM will build the world's first large-scale, fault-tolerant quantum computer. <https://www.ibm.com/quantum/blog/large-scale-ftqc>. Accessed: 2025-08-19.
 - [21] Infocomm Media Development Authority (IMDA). 2023. Singapore Launches Southeast Asia's First Quantum-Safe Network Infrastructure. <https://www.imda.gov.sg/resources/press-releases-factsheets-and-speeches/press-releases/2023/sg-launches-southeast-asias-first-quantum-safe-network-infrastructure>. Accessed: 2025-04-03.
 - [22] Juniper Networks. 2022. Validation of Quantum-Safe MACsec – White Paper. <https://www.juniper.net/content/dam/www/assets/white-papers/us/en/2022/validation-of-quantum-safe-macsec-white-paper.pdf>. Accessed: 2025-04-03.
 - [23] Sergejs Kozlovičs, Krišjānis Petručeņa, Dāvis Lāriņš, and Juris Viksna. 2023. Quantum Key Distribution as a Service and Its Injection into TLS. In *Information Security Practice and Experience*, Weizhi Meng, Zheng Yan, and Vincenzo Piuri (Eds.). Springer Nature Singapore, Singapore, 527–545.
 - [24] Yingbin Liang, H Vincent Poor, Shlomo Shamai, et al. 2009. Information theoretic security. *Foundations and Trends® in Communications and Information Theory* 5, 4–5 (2009), 355–580.
 - [25] National Institute of Standards and Technology. 2024. *Module-Lattice-Based Digital Signature Standard*. Federal Information Processing Standards Publication FIPS 204. U.S. Department of Commerce, Washington, D.C. doi:10.6028/NIST.FIPS.204
 - [26] National Institute of Standards and Technology. 2024. *Module-Lattice-Based Key Encapsulation Mechanism Standard*. Federal Information Processing Standards Publication FIPS 203. U.S. Department of Commerce, Washington, D.C. doi:10.6028/NIST.FIPS.203
 - [27] NodeQ Inc. 2024. PQTunnel: Seamless Post-Quantum IP Tunneling. White paper, NodeQ. <https://www.nodeq.io/pqtunnel>
 - [28] Nokia Corporation. 2024. Nokia and SK Broadband Deploy Quantum-Safe Network to Protect Korea Hydro and Nuclear Power's IT Infrastructure. <https://quantumcomputingreport.com/nokia-and-sk-broadband-deploy-quantum-safe-network-to-protect-korea-hydro-and-nuclear-powers-it-infrastructure/>. Accessed: 2025-04-03.
 - [29] Open Quantum Safe Project. 2024. liboqs. <https://openquantumsafe.org/liboqs/>. Accessed: 2025-08-07.
 - [30] pQCEe Research Group. 2024. QKDLite. <https://pqcee.github.io/QKDLite-manual/overview.html>
 - [31] Quantum Delta NL. 2023. Quantum Networks. <https://quantumdelta.nl/catalyst-programs/quantum-network>. Accessed: 2025-04-30.
 - [32] R. L. Rivest, A. Shamir, and L. Adleman. 1978. A method for obtaining digital signatures and public-key cryptosystems. *Commun. ACM* 21, 2 (Feb. 1978), 120–126. doi:10.1145/359340.359342
 - [33] Peter W Shor. 1994. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th annual symposium on foundations of computer science*. Ieee, 124–134.
 - [34] Peter W. Shor. 1997. Fault-tolerant quantum computation. arXiv:quant-ph/9605011 [quant-ph] <https://arxiv.org/abs/quant-ph/9605011>
 - [35] Peter W Shor and John Preskill. 2000. Simple proof of security of the BB84 quantum key distribution protocol. *Physical review letters* 85, 2 (2000), 441.
 - [36] Singtel. 2025. Quantum-Safe Network. <https://www.singtel.com/business/products-services/quantumsafenetwork>. Accessed: 2025-04-30.
 - [37] Anthony Spadafora. 2024. *What is Q-Day?*. Accessed: 2025-08-19.
 - [38] Splunk. 2023. Cryptographically Relevant Quantum Computers (CRQCs) & The Quantum Threat. https://www.splunk.com/en_us/blog/learn/crqcs-cryptographically-relevant-quantum-computers.html. Accessed: 2025-08-07.
 - [39] SPTel. 2025. Quantum-Safe Network. <https://sptel.com/our-technology/quantum-safe-network/>. Accessed: 2025-04-30.
 - [40] Thales Group and PQCEE. 2024. *Securing the QKD-HSM Connectivity by PQCEE & Thales*. Technical Report. Thales Group. https://cpl.thalesgroup.com/sites/default/files/content/solution_briefs/securing-the-qkd-hsm-connectivity-by-pqcee-thales-sb.pdf
 - [41] The Korea Herald. 2024. SK Broadband, Nokia deploy quantum-secure network for Korea Hydro & Nuclear Power. <https://m.koreaherald.com/article/3202176>. Accessed: 2025-04-03.
 - [42] UK Research and Innovation. 2025. UK National Quantum Technologies Programme. <https://uknqt.ukri.org/>. Accessed: 2025-04-30.
 - [43] VentureRadar. 2024. Top Quantum Key Distribution Companies. <https://www.ventureradar.com/keyword/Quantum%20Key%20Distribution>. Accessed: 2025-04-30.
 - [44] William K Wootters and Wojciech H Zurek. 1982. A single quantum cannot be cloned. *Nature* 299, 5886 (1982), 802–803.

A Benchmarking Results

Table 2 reports the performance benchmarking results of four protocols—Baseline TLS 1.2, ML-KEM-1024, FrodoKEM-1344-SHAKE, and BIKE-L5—under varying concurrency levels ranging from 10 to 500. For each configuration, the table presents throughput (requests per second), average latency, median latency (p50), tail latency (p99), the total number of processed requests, and the number of errors.

As expected, Baseline TLS 1.2 consistently achieves the highest throughput and lowest latencies across all concurrency levels, serving as a reference point for comparison. Among the PQC-enabled protocols, ML-KEM-1024 demonstrates relatively strong performance, with throughput values close to the baseline and moderate latency overheads. FrodoKEM-1344-SHAKE exhibits higher latency, particularly at larger concurrency, reflecting its computational cost. BIKE-L5 shows the steepest performance drop, with significantly lower throughput and higher latencies under high concurrency, though it remains error-free.

Overall, the results highlight the trade-off between post-quantum security strength and system performance, where PQC integration incurs additional latency and throughput penalties compared to classical TLS, with protocol-specific variations.

Table 2: Benchmark results of four protocols under varying concurrency levels

Protocol	Concurrency	Requests/s	Avg Latency (ms)	p50 (ms)	p99 (ms)	Total Requests	Errors
Baseline TLSv1.2	10	7197.46	1.18	1.00	4.93	216035	0
	25	6502.59	4.31	3.09	29.42	195318	0
	50	7771.24	6.14	6.05	9.31	233373	0
	100	7342.59	13.89	12.75	47.84	220616	0
	200	7607.44	25.10	25.41	34.25	228707	0
	500	6551.89	82.19	67.08	420.47	197161	0
ML-KEM-1024	10	5258.47	1.56	1.43	5.21	157933	0
	25	5134.00	5.71	4.22	37.47	154314	0
	50	5502.54	8.69	8.47	15.81	165284	0
	100	5200.37	22.04	17.08	125.92	156489	0
	200	5737.99	34.32	34.00	52.43	172606	0
	500	5114.39	95.79	88.38	298.95	153938	0
FrodoKEM-1344-SHAKE	10	4899.28	1.78	1.48	10.24	147189	0
	25	4339.06	8.58	4.44	82.71	130580	0
	50	5201.52	10.03	8.58	54.91	156535	0
	100	4854.54	23.46	17.82	144.80	146096	0
	200	5082.13	35.20	35.22	48.85	152955	0
	500	3930.38	95.67	91.47	331.42	118174	0
BIKE-L5	10	5122.19	1.70	1.44	8.14	153875	0
	25	4742.59	6.67	4.47	55.96	142650	0
	50	5314.90	9.50	8.55	46.11	159827	0
	100	5058.49	21.76	17.56	135.66	152172	0
	200	5235.70	34.00	33.99	53.97	157458	0
	500	3376.78	122.20	93.46	605.88	101604	0